Les tests fonctionnels avec WebTestCase et Panther

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Exploiter WebTestCase et HttpClient pour les tests fonctionnels avec Symfony **3**

A. Exploiter WebTestCase et HttpClient pour les tests fonctionnels avec Symfony 3

B. Exercice : Quiz 7

III. Test fonctionnel et automatisation avec Client, Crawler et Panther **8**

A. Test fonctionnel et automatisation avec Client, Crawler et Panther 8

B. Exercice : Quiz 11

IV. Essentiel **12**

V. Auto-évaluation **12**

A. Exercice 12

B. Test 13

Solutions des exercices **13**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 h

Environnement de travail : PC ou Mac avec 8 Go de RAM minimum

Prérequis : Notion PHP, Symfony

Contexte

Imaginez le processus de création d'un produit. Après de longues heures de travail, votre application est enfin prête. Vous êtes satisfait du résultat et impatient de le mettre entre les mains de vos utilisateurs. Cependant, une fois lancée, vous vous rendez compte que certaines fonctionnalités ne fonctionnent pas comme prévu, provoquant des frustrations pour vos utilisateurs et potentiellement, des pertes financières pour votre entreprise.

Avec des tests fonctionnels appropriés, vous pouvez éviter ce scénario. Les tests fonctionnels vous permettent de vérifier que votre application fonctionne comme prévu, simulant les actions que vos utilisateurs pourraient effectuer et vérifiant que le résultat est celui attendu.

Si vous travaillez avec Symfony, vous disposez d'outils pour effectuer ces tests fonctionnels, tels que le Client, le Crawler et Panther. Ces outils vous permettent de tester votre application de manière extensive, allant des interactions simples aux scénarios impliquant du JavaScript, garantissant que votre application est prête à être lancée.

Dans ce cours, nous plongerons dans le monde des tests fonctionnels avec Symfony, en explorant les fonctionnalités de ces outils et en apprenant comment vous pouvez les utiliser pour améliorer la qualité de votre application.

II. Exploiter WebTestCase et HttpClient pour les tests fonctionnels avec Symfony

A. Exploiter WebTestCase et HttpClient pour les tests fonctionnels avec Symfony

Les tests fonctionnels

Un test fonctionnel est centré sur une user story, une description détaillée du comportement d'une fonctionnalité à travers les yeux de l'utilisateur. Par exemple, une user story pourrait décrire comment un utilisateur ajoute un produit à son panier et effectue un achat. Pour réaliser ces tests, la simulation d'un client web est essentielle, car elle permet de reproduire les interactions réelles d'un utilisateur avec votre application. Des outils comme le Client et le Crawler de Symfony sont particulièrement utiles pour cela. Mais faire un bon test fonctionnel requiert aussi de suivre certaines règles. Chaque test doit être indépendant des autres, évitant ainsi qu'une erreur dans un test n'affecte les autres. Ils doivent aussi être aussi simples que possible, facilitant l'identification des problèmes. Et surtout, chaque test doit vérifier une seule fonctionnalité ou un seul comportement à la fois. Ce niveau de détail assure une couverture de test précise et facilite le diagnostic et la correction des problèmes, assurant ainsi que votre application fonctionne comme prévu.

Introduction à WebTestCase

WebTestCase est une classe spéciale fournie par Symfony qui est conçue pour faciliter les tests fonctionnels de votre application web. Cette classe est dérivée de la classe TestCase de PHPUnit, ce qui signifie qu'elle hérite de toutes les fonctionnalités de PHPUnit tout en ajoutant des fonctionnalités spécifiques à Symfony.

Le rôle principal de WebTestCase est de simuler le comportement d'un navigateur. Grâce à cette classe, nous pouvons envoyer des requêtes à notre application, cliquer sur des liens, soumettre des formulaires, et même utiliser des sessions et des cookies, tout cela directement dans nos tests.

L'importance de WebTestCase réside dans sa capacité à nous permettre de tester notre application du point de vue de l'utilisateur. Cela nous aide à vérifier que toutes les pièces de notre application fonctionnent ensemble comme prévu, ce qui est crucial pour garantir une expérience utilisateur optimale.

Méthode

L'utilisation de base de WebTestCase consiste à créer une nouvelle classe de test qui étend WebTestCase, puis à écrire des méthodes de test dans cette classe. Chaque méthode de test doit représenter un scénario fonctionnel que vous voulez tester dans votre application.

Voici un exemple de base de l'utilisation de WebTestCase pour tester une page d'accueil :

```php
namespace Tests;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class HomepageTest extends WebTestCase
{
    public function testHomePageIsDisplayedCorrectly()
    {
        // Crée un nouveau "client" (simulateur de navigateur)
        $client = static::createClient();

        // Utilise le client pour envoyer une requête à la page d'accueil
        $crawler = $client->request('GET', '/');

        // Vérifie que la requête a réussi (code de statut HTTP 200)
        $this->assertResponseIsSuccessful();

        // Vérifie que le contenu de la page est correct
        $this->assertSelectorTextContains('h1', 'Bienvenue sur notre site web !');
    }
}
```

Ce test s'appelle un SmokeTest. Le but d'un SmokeTest est de s'assurer que chaque route de l'application retourne un code 2.X.X ou 3.X.X.

Introduction à HttpClient

HttpClient est une composante puissante de Symfony qui facilite l'envoi de requêtes HTTP et la réception de réponses HTTP dans votre application. Il s'agit essentiellement d'un outil qui vous permet d'interagir avec d'autres sites web ou services web au sein de votre application.

HttpClient est conçu pour être très flexible et facile à utiliser. Il prend en charge de nombreux aspects des requêtes HTTP, y compris les cookies, les redirections, le streaming et même le téléchargement de fichiers. HttpClient est également capable de gérer simultanément plusieurs requêtes HTTP, ce qui peut améliorer considérablement les performances de votre application lors de l'interaction avec plusieurs services web.

Tous droits réservés à STUDI - Reproduction interdite

Méthode

L'utilisation de base de HttpClient consiste à créer un client HttpClient, à envoyer une requête HTTP avec ce client, puis à examiner la réponse.

Voici un exemple d'utilisation de HttpClient pour envoyer une requête GET à un site web :

```php
namespace App\Service;

use Symfony\Contracts\HttpClient\HttpClientInterface;

class MyService
{
    private $client;

    public function __construct(HttpClientInterface $client)
    {
        $this->client = $client;
    }

    public function getWebsiteContent()
    {
        // Envoyer une requête GET à un site web
        $response = $this->client->request('GET', 'https://www.example.com');

        // Vérifier que la requête a réussi
        $statusCode = $response->getStatusCode();
        // $statusCode doit être 200 si tout s'est bien passé

        // Récupérer le contenu de la réponse
        $content = $response->getContent();
        // $content contient le contenu du site web

        return $content;
    }
}
```

Ici, nous injectons HttpClientInterface dans notre service, puis nous utilisons cet objet client pour envoyer une requête GET à www.example.com[1]. Nous vérifions que la requête a réussi en examinant le code de statut HTTP, puis nous récupérons le contenu de la réponse avec getContent().

Les méthodes de Client

L'objet Client dans Symfony est au cœur des tests fonctionnels, agissant comme une passerelle entre votre test et votre application. Il simule un navigateur qui peut interagir avec votre application, en envoyant des requêtes HTTP à celle-ci et en recevant des réponses. Pour effectuer ces actions, l'objet Client fournit un certain nombre de méthodes qui vous permettent de contrôler son comportement.

Méthode **Méthode "request"**

La méthode "request" de la classe Client de Symfony est utilisée pour envoyer une requête HTTP à votre application. La syntaxe de base de cette méthode est la suivante :

```php
$client->request(string $method, string $uri, array $parameters = [], array $files = [], array $server = [], string|null $content = null, bool $changeHistory = true): Crawler
```

1 http://www.example.com/

Tous droits réservés à STUDI - Reproduction interdite

Le code PHP fourni permet d'effectuer une requête en utilisant la méthode request d'un objet $client. Cette méthode prend plusieurs paramètres. Tout d'abord, le paramètre $method représente la méthode HTTP que vous souhaitez utiliser, comme par exemple 'GET', 'POST', etc. Ensuite, le paramètre $uri correspond à l'URI vers laquelle vous souhaitez envoyer la requête. Vous pouvez également spécifier des paramètres supplémentaires à l'aide du tableau $parameters. Ces paramètres seront inclus dans la requête. Si vous avez des fichiers à envoyer avec la requête, vous pouvez les spécifier en utilisant le tableau $files.

Le tableau $server vous permet de définir des variables du serveur qui seront utilisées lors de la requête. Vous pouvez également inclure un contenu spécifique dans le corps de la requête en utilisant la variable $content.

Enfin, le dernier paramètre $changeHistory est un booléen qui indique si cette requête doit modifier l'historique du navigateur.

Voici un exemple d'utilisation de la méthode "request" pour envoyer une requête GET à la page d'accueil ainsi que d'un envoi POST avec des paramètres :

```php
$crawler = $client->request('GET', '/');

$crawler = $client->request('POST', '/submit', ['username' => 'Alice', 'password' => 'password123']);
```

Méthode **Les méthodes "back", "forward", "reload", "click"**

Introduisons quelques méthodes ensemble :

- La méthode "back" permet de naviguer vers la page précédente, tout comme le bouton "Précédent" d'un navigateur. La syntaxe est $client->back().
- La méthode "forward" permet de naviguer vers la page suivante, tout comme le bouton "Suivant" d'un navigateur. La syntaxe est $client->forward().
- La méthode "reload" recharge la page actuelle. La syntaxe est $client->reload().
- La méthode "click" permet de cliquer sur un lien. Elle prend en argument un objet Link que vous pouvez obtenir à partir d'un objet Crawler. La syntaxe est $client->click(Link $link).

Voici comment vous pourriez utiliser ces méthodes :

```php
// Aller à la page d'accueil
$crawler = $client->request('GET', '/');

// Cliquer sur le premier lien de la page
$link = $crawler->filter('a')->link();
$crawler = $client->click($link);

// Recharger la page
$crawler = $client->reload();

// Aller à la page précédente
$crawler = $client->back();

// Aller à la page suivante
$crawler = $client->forward();
```

Il existe d'autres méthodes importantes de la classe Client que vous pouvez utiliser dans vos tests.

Tous droits réservés à STUDI - Reproduction interdite

Par exemple :

- submit(Form $form, array $values = []) : permet de soumettre un formulaire.
- followRedirect() : suit la redirection si la dernière réponse était une redirection.
- getCookieJar() : obtient l'objet CookieJar qui contient tous les cookies actuels.
- getHistory() : obtient l'objet History qui contient l'historique de navigation.
- getServerParameters() : obtient les paramètres du serveur qui sont utilisés pour chaque requête.

B. Exercice : Quiz

[solution n°1 p.15]

Question 1

Qu'est-ce que WebTestCase dans Symfony ?

- Une classe pour effectuer des tests fonctionnels sur les applications web
- Une interface pour gérer les tests unitaires
- Un outil pour interagir avec la base de données

Question 2

Comment créer un nouveau "client" en utilisant WebTestCase ?

- Avec createClient()
- Avec createHttpClient()
- Avec createWebClient()

Question 3

Qu'est-ce que HttpClient dans Symfony ?

- Un outil pour exécuter du JavaScript dans les tests
- Un outil qui offre une interface pour simuler des requêtes HTTP et examiner la réponse
- Un outil pour tester l'interface utilisateur d'une application web

Question 4

Quelle méthode permet de vérifier le code de statut HTTP de la réponse avec HttpClient ?

- La méthode getStatus()
- La méthode getCode()
- La méthode getStatusCode()

Question 5

Quelle est l'utilité de la méthode request de la classe Client de Symfony ?

- Envoyer une requête HTTP à l'application
- Soumettre un formulaire
- Charger un fichier

Tous droits réservés à STUDI - Reproduction interdite

III. Test fonctionnel et automatisation avec Client, Crawler et Panther

A. Test fonctionnel et automatisation avec Client, Crawler et Panther

Configurer le client

Lors de l'écriture de tests fonctionnels avec Symfony, il est souvent nécessaire de configurer l'objet Client pour simuler différentes conditions de test. Cette configuration peut comprendre des éléments tels que l'ensemble de l'environnement de l'application, l'adresse IP du client, le User Agent HTTP, les informations d'authentification, et bien d'autres.

Ces configurations peuvent être essentielles pour garantir que vos tests couvrent une gamme suffisamment large de scénarios réalistes. Par exemple, vous pouvez vouloir tester comment votre application se comporte lorsqu'elle est exécutée dans l'environnement de production par rapport à l'environnement de test, ou comment elle réagit à des requêtes provenant de différents navigateurs ou adresses IP.

C'est pour cela que nous allons explorer en détail comment configurer l'objet Client dans Symfony, en passant en revue les différentes options disponibles et en montrant des exemples de leur utilisation.

Méthode **Comment configurer l'environnement ("env")**

L'environnement de votre application peut être configuré lors de la création de l'objet client. Vous pouvez spécifier l'environnement en utilisant le paramètre env dans la méthode createClient().

Pour créer un client dans l'environnement de test, voici comment procéder :

```
1 $client = static::createClient([], ['APP_ENV' => 'test']);
```

Méthode **Configuration de "remote_addr"**

L'adresse IP du client peut être configurée en utilisant le paramètre HTTP_CLIENT_IP ou REMOTE_ADDR lors de la création de l'objet client.

```
1 $client = static::createClient([], ['HTTP_CLIENT_IP' => '192.168.1.1', 'REMOTE_ADDR' =>
  '192.168.1.1']);
```

Méthode **Configuration de "http_user_agent"**

Le User Agent HTTP peut être configuré en utilisant le paramètre HTTP_USER_AGENT lors de la création de l'objet client.

Pour simuler un client utilisant Google Chrome, nous écrirons ceci :

```
1 $client = static::createClient([], ['HTTP_USER_AGENT' => 'Mozilla/5.0 (Windows NT 10.0; Win64;
  x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537']);
```

Méthode **Configuration de l'authentification**

L'authentification HTTP peut être configurée en utilisant les paramètres PHP_AUTH_USER et PHP_AUTH_PW lors de la création de l'objet client.

```
1 $client = static::createClient([], ['PHP_AUTH_USER' => 'username', 'PHP_AUTH_PW' =>
  'password']);
```

Notez que vous pouvez privilégier l'utilisation d'un User créé par fixtures dans un environnement de test.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Autres configurations essentielles**

Il y a d'autres paramètres importants que vous pouvez configurer lors de la création de l'objet client. Par exemple, vous pouvez configurer le serveur HTTP en utilisant le paramètre HTTP_HOST, ou vous pouvez configurer le port en utilisant le paramètre HTTP_PORT. Vous pouvez également configurer d'autres en-têtes HTTP en utilisant les paramètres HTTP_*.

En général, la configuration de l'objet client dépend des besoins spécifiques de vos tests. Il est recommandé de consulter la documentation de Symfony pour une liste complète des options disponibles.

Introduction au Crawler

Le Crawler - fourni par Symfony - permet de naviguer dans le DOM (Document Object Model) de votre page web. Cela peut être particulièrement utile pour extraire des informations d'une page ou pour interagir avec celle-ci lors des tests fonctionnels. Par exemple, vous pouvez utiliser le Crawler pour trouver des éléments spécifiques dans la page, cliquer sur des liens, remplir des formulaires, etc.

En fait, si vous avez suivi les exemples de ce cours jusqu'à présent, vous avez déjà utilisé le Crawler sans même le savoir. Chaque fois que vous utilisez la méthode request de l'objet Client pour envoyer une requête à votre application, cette méthode renvoie un objet Crawler que vous pouvez utiliser pour interagir avec la page.

Méthode

Supposons que nous ayons une page web avec plusieurs paragraphes et liens, et que nous voulions vérifier le contenu du premier et du dernier paragraphe puis cliquer sur le deuxième lien.

Voici comment nous pourrions le faire :

```php
$crawler = $client->request('GET', '/');

// Trouver le premier paragraphe et vérifier son contenu
$firstParagraph = $crawler->filter('p')->first();
$this->assertEquals('Expected content of the first paragraph', $firstParagraph->text());

// Trouver le dernier paragraphe et vérifier son contenu
$lastParagraph = $crawler->filter('p')->last();
$this->assertEquals('Expected content of the last paragraph', $lastParagraph->text());

// Trouver le deuxième lien et cliquer dessus
$secondLink = $crawler->filter('a')->eq(1);
$crawler = $client->click($secondLink->link());

// Vérifier que nous avons été redirigés vers la bonne page
$this->assertEquals('/expected/url', $client->getRequest()->getPathInfo());
```

Dans notre exemple, nous utilisons le Crawler pour naviguer à travers une page web et interagir avec elle de manière similaire à un utilisateur. Nous commençons par envoyer une requête GET à la page en utilisant la méthode request de l'objet Client. Cette méthode renvoie un objet Crawler qui représente la page. Nous utilisons ensuite la méthode filter() du Crawler pour trouver tous les paragraphes de la page. Cette méthode prend un sélecteur CSS en paramètre et renvoie tous les éléments qui correspondent à ce sélecteur.

Pour extraire le contenu du premier paragraphe, nous utilisons la méthode first du Crawler. Cette méthode renvoie le premier élément de la liste des éléments trouvés. Ensuite, nous utilisons la méthode text pour obtenir le contenu textuel de cet élément. Nous faisons la même chose pour le dernier paragraphe, en utilisant la méthode last qui renvoie le dernier élément de la liste.

Nous trouvons ensuite le deuxième lien sur la page en utilisant la méthode eq, qui renvoie l'élément à un index spécifique dans la liste. En passant l'index 1 à cette méthode, nous obtenons le deuxième lien (puisque les indices commencent à 0).

Tous droits réservés à STUDI - Reproduction interdite

Enfin, nous utilisons la méthode click de l'objet Client pour simuler un clic sur ce lien. Cette méthode envoie une nouvelle requête à l'URL de ce lien et renvoie un nouvel objet Crawler qui représente la page résultante. Nous vérifions ensuite que nous avons été redirigés vers la bonne URL en utilisant la méthode getRequest de l'objet Client pour obtenir la requête actuelle, puis en appelant la méthode getPathInfo sur cette requête pour obtenir le chemin de l'URL.

Introduction à Panther et PantherClient

Panther est un outil de test de navigateur open source développé par Kévin Dunglas qui permet d'automatiser les interactions avec votre application web, tout comme le ferait un utilisateur réel. Il est particulièrement utile pour les tests d'acceptation et d'intégration et prend en charge les applications JavaScript, grâce à sa capacité à piloter un véritable navigateur.

Panther offre une classe spéciale appelée PantherClient qui est une extension du Client HTTP traditionnel de Symfony. La principale différence réside dans la capacité de PantherClient à interagir avec du JavaScript, ce qui n'est pas possible avec le client HTTP traditionnel.

Les applications modernes font souvent un usage intensif de JavaScript pour dynamiser l'interface utilisateur. Par conséquent, si vous voulez tester ces fonctionnalités, vous avez besoin d'un outil qui peut interagir avec JavaScript. PantherClient est cet outil : il utilise un véritable navigateur (Chrome par défaut, mais Firefox est également supporté) pour interagir avec votre application, ce qui lui permet d'exécuter du JavaScript et d'interagir avec des éléments de la page qui sont manipulés par JavaScript.

Méthode **Mise en place et utilisation de PantherClient**

La première étape pour utiliser PantherClient est d'installer Panther. Vous pouvez le faire en utilisant Composer :

```
$ composer require --dev symfony/panther
```

Ensuite, dans vos tests, vous devez créer une classe qui hérite de PantherTestCase. Cette classe parente fournit la méthode createPantherClient() pour instancier un PantherClient.

```
namespace Tests;

use Symfony\Component\Panther\PantherTestCase;

class MyTest extends PantherTestCase
{
   public function testMyFeature()
   {
       $client = static::createPantherClient();
       // ...
   }
}
```

La partie View testée avec Panther

Les tests de vue constituent une partie importante des tests fonctionnels. Ils vous permettent de vérifier que votre application rend les pages correctement, que les bons éléments sont présents et que les utilisateurs peuvent interagir avec eux comme prévu.

Méthode

Grâce à son intégration avec un véritable navigateur, Panther est idéalement conçu pour les tests de vue. Avec Panther, vous pouvez vérifier le contenu d'une page, cliquer sur des liens, remplir et soumettre des formulaires, et même interagir avec des éléments de la page qui sont manipulés par JavaScript.

Tous droits réservés à STUDI - Reproduction interdite

Supposons que nous souhaitons tester le processus d'inscription d'un utilisateur. Nous voulons naviguer vers la page d'inscription, remplir le formulaire, le soumettre, et ensuite vérifier que nous sommes redirigés vers la page de confirmation qui affiche un message de réussite.

```php
public function testRegistration()
{
    $client = static::createPantherClient();

    // Navigate to the registration page
    $crawler = $client->request('GET', '/register');

    // Fill in the form and submit it
    $form = $crawler->selectButton('Submit')->form([
        'username' => 'testuser',
        'password' => 'testpass',
    ]);
    $client->submit($form);

    // Check that we are redirected to the confirmation page
    $this->assertSame('/confirmation', $client->getRequest()->getUri());

    // Check that the confirmation page shows a success message
    $this->assertContains('Registration successful', $client->getResponse()->getContent());
}
```

Cet exemple de test illustre comment Panther peut être utilisé pour tester une interaction utilisateur complexe impliquant plusieurs requêtes, la soumission d'un formulaire et des interactions avec du JavaScript.

B. Exercice : Quiz

[solution n°2 p.16]

Question 1

À quoi sert le Crawler dans le contexte des tests fonctionnels de Symfony ?

- Il permet de parcourir et manipuler le DOM d'une réponse HTTP
- Il sert à extraire les liens d'une page
- Il sert à simuler des requêtes HTTP

Question 2

Que signifie le terme "Panther" dans Symfony Panther ?

- Il fait référence à un outil pour les tests de performance
- Il fait référence à un outil pour automatiser les interactions avec votre application web
- Il fait référence à un outil pour les tests unitaires

Question 3

Comment pouvez-vous vérifier la présence d'un élément spécifique dans le DOM avec Crawler ?

- Avec la méthode findElement()
- Avec la méthode search()
- Avec la méthode filter()

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Comment pouvez-vous récupérer le contenu d'une réponse HTTP avec l'objet Client de Symfony ?

- Avec getContent()
- Avec getResponse()
- Avec retrieve()

Question 5

Comment créer un PantherClient ?

- En instanciant directement la classe PantherClient
- En utilisant lcreateClient() de la classe WebTestCase
- En utilisant createPantherClient() de la classe PantherTestCase

IV. Essentiel

Dans ce cours sur les tests fonctionnels et l'automatisation avec Symfony, nous avons abordé des notions essentielles pour configurer le client, utiliser le crawler et Panther.

Nous avons appris que la configuration du client permettait de simuler différentes conditions de test, telles que l'environnement, l'adresse IP du client, le User Agent HTTP et l'authentification. De plus, en utilisant la méthode createClient(), nous pouvons spécifier l'environnement et les paramètres tels que HTTP_CLIENT_IP, REMOTE_ADDR et HTTP_USER_AGENT.

Le crawler, fourni par Symfony, permet quant à lui de naviguer dans le DOM d'une page web pour extraire des informations ou interagir avec celle-ci lors des tests fonctionnels. Chaque fois que nous utilisons la méthode request() du client, elle renvoie un objet Crawler pour interagir avec la page.

Panther, un outil open source, offre PantherClient qui étend le client HTTP de Symfony et permet d'automatiser les interactions avec une application web, y compris le JavaScript. PantherClient utilise un véritable navigateur pour exécuter du JavaScript et interagir avec des éléments manipulés par JavaScript.

En suivant ce cours, vous avez parcouru un chemin impressionnant. Votre engagement et votre curiosité vous ont conduit à travers les défis et les complexités des tests fonctionnels dans Symfony. Vous avez plongé profondément dans des concepts nouveaux et exigeants, vous avez fait preuve de patience et de persévérance, et vous avez continué à apprendre et à évoluer tout au long de ce processus.

V. Auto-évaluation

A. Exercice

Vous développez un site web pour une librairie en ligne. Vous avez une route "/submit" qui accepte une requête POST avec un nom de livre et son prix pour ajouter un livre à la librairie. Vous voulez tester cette fonctionnalité en utilisant l'objet Client de Symfony.

Question 1 [solution n°3 p.17]

Créez un test qui envoie une requête POST à la route "/submit" avec un nom de livre et son prix, puis vérifiez que la réponse à un statut HTTP de 200.

Vous avez une page d'accueil qui affiche les livres les plus vendus de votre librairie en ligne. Chaque livre est représenté par un élément avec la classe "book". Vous voulez tester que le nombre de livres affichés sur la page d'accueil est correct.

Question 2 [solution n°4 p.17]

Écrivez un test qui navigue vers la page d'accueil, utilisez le Crawler pour compter le nombre d'éléments "book" sur la page, et vérifiez que ce nombre est égal au nombre de livres les plus vendus que vous attendez.

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°5 p.17]

Question 1

Comment naviguer vers la page précédente avec l'objet Client de Symfony ?

- Avec previous()
- Avec back()
- Avec goBack()

Question 2

Comment utilise-t-on la méthode request pour envoyer une requête POST avec des paramètres ?

- $client->request('POST', '/submit', ['username' => 'Alice', 'password' => 'password123']);
- $client->request('POST', '/submit', ['username', 'Alice', 'password', 'password123']);
- $client->request('POST', '/submit'); $client->setParameter('username', 'Alice'); $client->setParameter('password', 'password123');

Question 3

Comment récupérer l'historique de navigation avec l'objet Client de Symfony ?

- Avec getHistory()
- Avec history()
- Avec getNavigationHistory()

Question 4

Comment réaliser une assertion sur le statut de la réponse HTTP avec Symfony ?

- En utilisant assertResponseIsSuccessful()
- En utilisant assertStatus()
- En utilisant assertStatusCode()

Question 5

Quelle méthode utiliser pour suivre une redirection avec l'objet Client de Symfony ?

- En utilisant follow()
- En utilisant followRedirect()
- En utilisant redirect()

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 7 Solution n°1

Question 1

Qu'est-ce que WebTestCase dans Symfony ?

- [x] Une classe pour effectuer des tests fonctionnels sur les applications web
- [ ] Une interface pour gérer les tests unitaires
- [ ] Un outil pour interagir avec la base de données

WebTestCase est une classe fournie par Symfony pour aider à écrire des tests fonctionnels pour les applications web. Elle offre des méthodes pour simuler des requêtes HTTP et examiner la réponse.

Question 2

Comment créer un nouveau "client" en utilisant WebTestCase ?

- [x] Avec createClient()
- [ ] Avec createHttpClient()
- [ ] Avec createWebClient()

Pour créer un nouveau "client" (simulateur de navigateur) avec WebTestCase il est nécessaire d'employer la méthode createClient().

Question 3

Qu'est-ce que HttpClient dans Symfony ?

- [ ] Un outil pour exécuter du JavaScript dans les tests
- [x] Un outil qui offre une interface pour simuler des requêtes HTTP et examiner la réponse
- [ ] Un outil pour tester l'interface utilisateur d'une application web

HttpClient est une bibliothèque fournie par Symfony qui apporte une interface pour simuler des requêtes HTTP, ce qui est essentiel pour les tests fonctionnels d'une application web.

Question 4

Quelle méthode permet de vérifier le code de statut HTTP de la réponse avec HttpClient ?

- [ ] La méthode getStatus()
- [ ] La méthode getCode()
- [x] La méthode getStatusCode()

Une fois qu'une requête a été envoyée à l'aide de HttpClient et qu'une réponse a été reçue, getStatusCode() vous permet d'obtenir le code de statut HTTP de la réponse.

Question 5

Quelle est l'utilité de la méthode request de la classe Client de Symfony ?

Tous droits réservés à STUDI - Reproduction interdite

- [x] Envoyer une requête HTTP à l'application
- [ ] Soumettre un formulaire
- [ ] Charger un fichier

La méthode request() de la classe Client de Symfony est utilisée pour simuler l'envoi d'une requête HTTP à votre application.

Exercice p. 11 **Solution n°2**

Question 1

À quoi sert le Crawler dans le contexte des tests fonctionnels de Symfony ?

- [x] Il permet de parcourir et manipuler le DOM d'une réponse HTTP
- [ ] Il sert à extraire les liens d'une page
- [ ] Il sert à simuler des requêtes HTTP

Le Crawler est utilisé pour naviguer dans le DOM de la réponse HTTP. Il permet d'extraire des informations, de manipuler le DOM et de tester la structure et le contenu de vos pages.

Question 2

Que signifie le terme "Panther" dans Symfony Panther ?

- [ ] Il fait référence à un outil pour les tests de performance
- [x] Il fait référence à un outil pour automatiser les interactions avec votre application web
- [ ] Il fait référence à un outil pour les tests unitaires

Symfony Panther est un outil qui permet d'automatiser les interactions avec votre application web, simulant ainsi l'expérience utilisateur réelle. Il est particulièrement utile pour les tests d'acceptation et d'intégration.

Question 3

Comment pouvez-vous vérifier la présence d'un élément spécifique dans le DOM avec Crawler ?

- [ ] Avec la méthode findElement()
- [ ] Avec la méthode search()
- [x] Avec la méthode filter()

La méthode filter() de Crawler est utilisée pour sélectionner des éléments dans le DOM en utilisant des sélecteurs CSS.

Question 4

Comment pouvez-vous récupérer le contenu d'une réponse HTTP avec l'objet Client de Symfony ?

- [x] Avec getContent()
- [ ] Avec getResponse()
- [ ] Avec retrieve()

La méthode getContent() de l'objet Client permet de récupérer le contenu de la réponse HTTP. Vous pouvez ainsi obtenir le corps de la réponse sous forme de chaîne.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Comment créer un PantherClient ?

- ○ En instanciant directement la classe PantherClient
- ○ En utilisant lcreateClient() de la classe WebTestCase
- ◉ En utilisant createPantherClient() de la classe PantherTestCase

Pour créer un PantherClient, nous utilisons la méthode createPantherClient() qui est requise pour instancier un PantherClient, car elle initialise correctement l'objet et s'assure qu'il est prêt à interagir avec une application web.

p. 12 **Solution n°3**

Voici une solution possible pour ce faire :

```php
public function testSubmitBook()
{
    $client = static::createClient();

    $client->request('POST', '/submit', [
        'book_name' => 'Le Petit Prince',
        'book_price' => '10.99',
    ]);

    $this->assertEquals(200, $client->getResponse()->getStatusCode());
}
```

Ici, l'objet Client est utilisé pour envoyer une requête POST à la route "/submit" puis nous passons un tableau associatif en paramètre à la méthode request(), ce qui représente les données POST. Ensuite, nous utilisons la méthode getStatusCode() sur l'objet Response pour obtenir le code de statut HTTP, et vérifions qu'il est égal à 200, ce qui signifie que la requête a réussi.

p. 12 **Solution n°4**

Voici une solution possible pour ce faire :

```php
public function testHomePageDisplaysBestSellingBooks()
{
    $client = static::createClient();

    $crawler = $client->request('GET', '/');

    $bookElements = $crawler->filter('.book');

    $this->assertEquals(5, $bookElements->count());
}
```

Ci-dessus, nous utilisons le Crawler pour interagir avec le DOM de la page d'accueil. Après avoir envoyé une requête GET à la route "/", nous utilisons la méthode filter() du Crawler pour obtenir tous les éléments DOM qui ont la classe "book". Ensuite, nous utilisons la méthode count() pour compter ces éléments. Enfin, nous vérifions que le nombre d'éléments correspond à notre attente (dans cet exemple, nous attendons 5 livres).

Exercice p. 13 **Solution n°5**

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Comment naviguer vers la page précédente avec l'objet Client de Symfony ?

- ○ Avec previous()
- ◉ Avec back()
- ○ Avec goBack()

Pour naviguer vers la page précédente vous pouvez utiliser la méthode back(). Cette fonction est pratique lorsque vous simulez des scénarios d'utilisation où l'utilisateur a besoin de revenir à la page précédente, par exemple après avoir consulté les détails d'un élément particulier.

Question 2

Comment utilise-t-on la méthode request pour envoyer une requête POST avec des paramètres ?

- ◉ $client->request('POST', '/submit', ['username' => 'Alice', 'password' => 'password123']);
- ○ $client->request('POST', '/submit', ['username', 'Alice', 'password', 'password123']);
- ○ $client->request('POST', '/submit'); $client->setParameter('username', 'Alice'); $client->setParameter('password', 'password123');

Pour envoyer une requête POST avec des paramètres à l'aide de l'objet Client de Symfony, vous utilisez la méthode request() avec le type de requête ('POST' dans ce cas), l'URL et un tableau associatif des paramètres. Dans l'exemple, 'username' et 'password' sont les clés du tableau, et 'Alice' et 'password123' sont leurs valeurs respectives.

Question 3

Comment récupérer l'historique de navigation avec l'objet Client de Symfony ?

- ◉ Avec getHistory()
- ○ Avec history()
- ○ Avec getNavigationHistory()

La méthode getHistory() permet de récupérer l'historique de navigation avec l'objet Client de Symfony. Cela peut être utile pour les tests qui nécessitent de vérifier que certaines actions de navigation ont été réalisées.

Question 4

Comment réaliser une assertion sur le statut de la réponse HTTP avec Symfony ?

- ◉ En utilisant assertResponseIsSuccessful()
- ○ En utilisant assertStatus()
- ○ En utilisant assertStatusCode()

Symfony fournit une série de méthodes d'assertion spécifiques pour les tests fonctionnels, parmi lesquelles assertResponseIsSuccessful(). Cette méthode vérifie si le statut de la réponse HTTP indique que la requête a été traitée avec succès.

Question 5

Quelle méthode utiliser pour suivre une redirection avec l'objet Client de Symfony ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ En utilisant follow()
- ◉ En utilisant followRedirect()
- ○ En utilisant redirect()

La méthode followRedirect() de l'objet Client permet de suivre une redirection. Elle est particulièrement utile dans les tests d'acceptation ou d'intégration où vous devez simuler le comportement de navigation d'un utilisateur réel.

Tous droits réservés à STUDI - Reproduction interdite